
Práctica 8: Render graph y pasadas en URP

Objetivo: Creación de pasadas simples con render graph en URP

1. Instrucciones

La práctica se realiza sobre un proyecto de Unity en el que se van añadiendo escenas con distintos elementos y que utilizan renderers distintos configurados para utilizar las *Render features* implementadas en cada apartado.

Se entregará el proyecto completo de Unity, dejando únicamente los directorios necesarios (Assets, Packages y ProjectSettings). En el directorio Assets habrá, además, una carpeta Capturas con imágenes de pantalla que muestren el resultado de cada una de las tareas pedidas. En este caso el resultado no es la escena final del juego sino el *Render graph viewer* donde se vea la ejecución de las pasadas implementadas (en el último apartado sí habrá que añadir además dos capturas).

La entrega tendrá un directorio Scripts en el que habrá cuatro directorios, uno por apartado, donde estará la implementación de cada pareja *Render feature* - *Render pass*.

2. Punto de partida

Crea un proyecto 3D en Unity que utilice el *Universal Render Pipeline*. Añade un directorio URPSettings para sus assets. Antes de empezar con la primera parte:

- Crea un asset “*URP Asset (with Universal Renderer)*” en la carpeta anterior llamado PR08_URenderPipelineAsset. Creará también un *renderer* que se utilizará por defecto PR08_URenderPipelineAsset_Renderer.
- Configura el proyecto (opción *Graphics* y quizá en las *Quality Settings*) para que utilice esa configuración.

La escena de cada apartado tendrá un renderer distinto, para lo que habrá que:

- Crear el nuevo renderer “*URP Universal Renderer*” con el nombre concreto indicado en la carpeta URPSettings.



Figura 1: Contenido del directorio Assets de la entrega

- Añadir el renderer en la lista de renderers de PR08_URenderPipelineAsset.
- Seleccionar ese renderer en la cámara de la escena.
- Aunque no es obligatorio, seguramente quieras poner el nuevo renderer como el renderer por defecto para que se utilice también en la pestaña *Scene* (si no has forzado el uso del renderer inicial en las primeras escenas, esta acción alterará su funcionamiento).

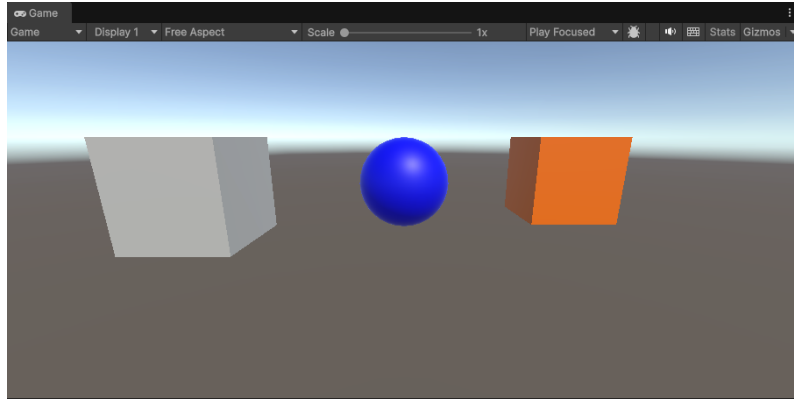
Crea en la carpeta Materials los siguientes materiales, basados en el shader por defecto de URP, Universal Render Pipeline/Lit:

- Azul que utilice el RGB (0, 0, 255).
- Naranja con (255, 120, 0).
- GrisClaro con (200, 200, 200).

Coloca en la escena los siguientes objetos:

- Cubo GrisClaro en L: (-2, 0.5, 0).
- Esfera Azul en L: (0, 0.5, 0.5).
- Cubo Naranja en L: (2, 0.5, 1).

Configura la cámara de la escena en L: (0, 1, -3) y R: (15, 0, 0).



3. *Copy pass*

Implementa una nueva *Render feature* y su pasada correspondiente que copie el buffer de color a una textura auxiliar, permitiendo configurar en el editor en qué momento se hace la copia.

Para eso, utiliza el método de extensión `AddCopyPass` de la clase `RenderGraph`. Hazlo de forma que la textura destino de la copia tenga como nombre fijo `CopyPass-Pr8`.

Hazlo en el directorio `Scripts/01CopyPass` utilizando los nombres de clases `CopyPassFeature` y `CopyPass`. Pruebalo en una escena llamada `01CopyPass` configurando el *renderer* utilizado en su cámara.

Incluye la captura del `Render Graph Viewer` en el que se vea.

4. Textura de color plano

Implementa una nueva *Renderer feature* y su pasada correspondiente que cree una textura auxiliar que se rellene con un color plano configurable desde el editor. La pasada se ejecutará tras renderizar los opacos y el nombre de la textura debe ser `ColorPlano-Pr8`.

Hazlo en el directorio `Scripts/02CreateColorTextureFeature` utilizando los nombres de clases `CreateColorTextureFeature` y `CreateColorTexturePass`. Pruebalo en una escena llamada `02CreateColorTextureFeature` configurando el *renderer* utilizado en su cámara.

Incluye la captura del `Render Graph Viewer` en el que se vea.

5. *Copy pass* sin ayuda

Implementa una nueva *Render feature* y su pasada correspondiente que copie el buffer de color a una textura auxiliar tras renderizar los opacos.

El funcionamiento será similar al de realizado en la sección 3, pero *sin hacer uso* del método de extensión `AddCopyPass`.

Para hacerlo utiliza el método `BlitTexture` de la clase `Blitter` pasándole los siguientes parámetros:

- El *command buffer*.
- La textura que hay que copiar.

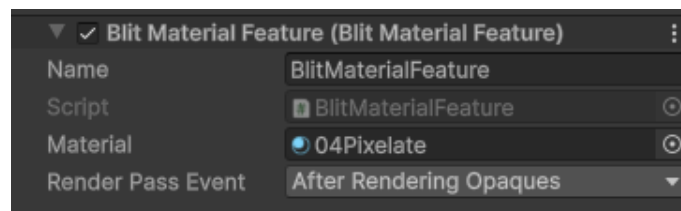
- Un `float4` con la escala y offset que se quiere utilizar y que será `(1, 1, 0, 0)` para utilizar la escala unitaria y sin offset.
- Un 0 indicando el mipmap level.
- Un `false` para que no se aplique interpolación al acceder a la textura.

Hazlo en el directorio `Scripts/03MyCopyPass` utilizando los nombres de clases `CopyPassFeature` y `CopyPass`. Pruebalo en una escena llamada `01CopyPass` configurando el *renderer* utilizado en su cámara.

Incluye la captura del Render Graph Viewer en el que se vea.

6. Blit con material

Crea una nueva *Render feature* y su pasada correspondiente que ejecute un `Blit` sobre el frame utilizando un material configurable y que permita ejecutar el efecto de pixelado de la práctica anterior después de renderizar los objetos opacos:



Para ello:

- Crea una nueva escena `04BlitMaterial` cuya cámara utilice un *renderer* nuevo con el mismo nombre.
- Crea la *feature* y pasada en dos ficheros que definan las clases `BlitMaterialFeature` y `BlitMaterialPass`.
- Crea un material nuevo, basado en el *Lit* de Unity que sea de color rojo transparente (con 128).
- Utiliza el material anterior en una nueva esfera colocada en L: `(0.5, 0.5, -1.5)`.

La implementación del *nodo* de `RenderGraph` se hará utilizando una función de extensión similar a `AddCopyPass` llamada `AddBlitPass` que debe renderizar a una textura llamada `BlitMaterial-Pr8`.

La función de extensión recibe un primer parámetro con una estructura que configura su comportamiento y un segundo parámetro con el nombre de la pasada (`Blit-Pr8`). La estructura del primer parámetro puede crearse con un código similar al siguiente (el 0 indica que se utilizará la primera pasada del material):

```
RenderGraphUtils.BlitMaterialParameters blitParams =
    new RenderGraphUtils.BlitMaterialParameters(
        source,
        destination,
        blitMaterial,
        0);
```

Incluye en la entrega:

- Una captura del *Render Graph Viewer*, igual que en las secciones anteriores.
- Una captura de la escena aplicando el efecto en el punto de inyección *After Rendering Post Processing* en la que toda la escena debería verse pixelada.
- Otra captura aplicando el efecto *After Rendering Opaques* de forma que sólo los opacos deben verse pixelados. Asegurate de que entiendes la anomalía que ocurre en sus bordes.

7. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- El proyecto de Unity con los directorios mínimos necesarios (`Assets`, `Packages` y `ProjectSettings`). El contenido del directorio `Assets` debe ser el que se deduce de las instrucciones anteriores y que puede verse en la figura 1.
- Un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.